ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

ассистент		В.В. Жукалин
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ № 3

Реализация клиентской части веб-приложения: фотографии, модальные окна, бургер-меню, состояние пользователя и карта

по курсу: Web-программирование

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4321		Г.В. Буренков
	подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

СОДЕРЖАНИЕ

1 Цель работы 3
2 Клиентский сценарий 4
3 Структура приложения 5
4 Сущность и фотографии 7
5 Галерея и модальное окно 9
5.1 Состояние и хуки 9
5.2 Доступность диалога 10
6 Бургер-меню 12
7 Карта 13
8 Результаты проверки 14
9 Ответы на контрольные вопросы 19
ЗАКЛЮЧЕНИЕ 22
ПРИЛОЖЕНИЕ А Листинги исходного кода 23

1 Цель работы

Цель работы — добавить в PairWise клиентскую часть: фотографии площадок, галерею с модальным окном, бургер-меню, действие избранного для учебного пользователя и карту с метками через JavaScript API Яндекс Карт.

2 Клиентский сценарий

Пользователь открывает раздел «Галерея и карта». На странице шесть карточек коворкингов Санкт-Петербурга. Щелчок по карточке открывает окно с фотографией, описанием и адресом. Гость в этом окне видит ссылку «Войдите, чтобы сохранить площадку». После учебного входа на том же месте появляется кнопка «Добавить в избранное». Повторное нажатие меняет подпись на «Убрать из избранного». Под сеткой написано, что избранное хранится только до обновления страницы.

Шапка открывает бургер-меню. В нём пункты «Главная», «Каталог», «Галерея и карта», «О проекте» и «Кабинет». Текущий раздел подсвечен. У гостя внизу панели ссылка «Войти». У учебного пользователя там имя и кнопка «Выйти». Ссылка «Вход» в правой части шапки остаётся на всех экранах и ведёт на /auth/login.

Под карточками расположен блок карты. Координаты площадок лежат в массиве данных. С ключом NEXT_PUBLIC_YANDEX_MAPS_API_KEY компонент создаёт карту Санкт-Петербурга и метку на каждую из шести площадок. Щелчок по метке открывает то же модальное окно, что и карточка: с метки открылась «ITMO Cowork». Если ключа в окружении нет, скрипт карт не запрашивается и на странице остаётся сообщение, что карта не подключена.

3 Структура приложения

Страница галереи живёт в app/gallery/page.tsx и открывается по адресу /gallery. Директивы «use client» в этом файле нет. Интерактивность вынесена в PlacesGallery, PhotoModal, PhotoMap и BurgerMenu. Шапка Header подключает бургер. Учебный пользователь по-прежнему хранится в DemoAuthProvider. Назначение файлов приведено в таблице 1, состав каталогов показан на рисунке 1.

Таблица 1 — Новые и изменённые части клиента

Назначение	**Файл**	**Где работает**
Страница галереи	app/gallery/page.tsx	/gallery
Сетка карточек и избранное	app/gallery/_components/PlacesGallery.tsx	клиент
Модальное окно	app/gallery/_components/PhotoModal.tsx	document.body
Карта	app/gallery/_components/PhotoMap.tsx	блок под сеткой
Бургер-меню	app/ui/BurgerMenu/BurgerMenu.tsx	шапка
Площадки, фото, координаты	app/data/spots.ts	каталог и галерея
Файлы изображений	public/images/spots	локальные jpg

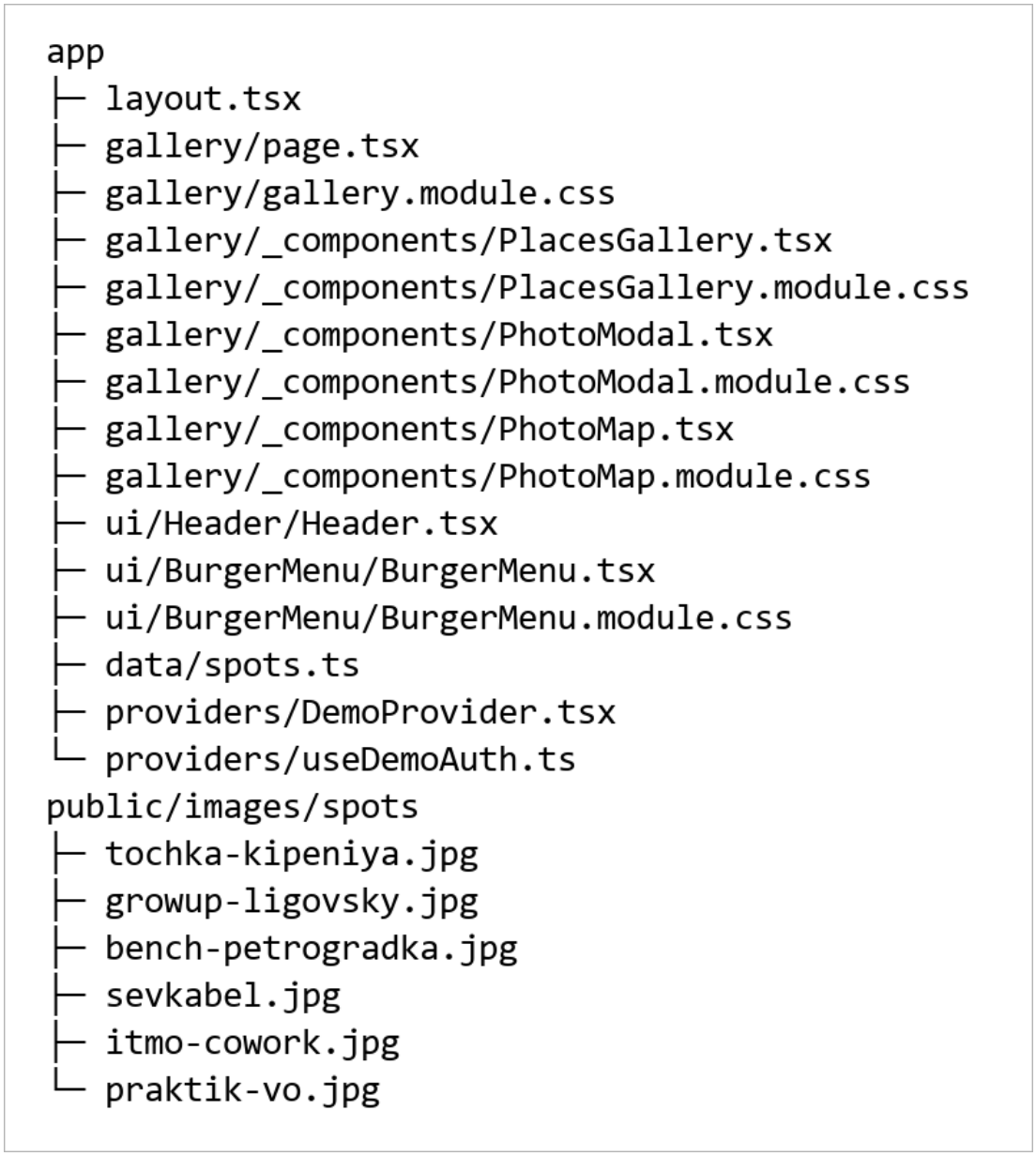


Рисунок 1 — Файлы клиентской части и фотографии площадок

4 Сущность и фотографии

Предметная сущность — площадка. К прежним полям добавлены путь к изображению, альтернативный текст, широта и долгота. Карта читает latitude и longitude. Поле coords сохранено: x совпадает с долготой, y — с широтой. В массиве шесть площадок с номерами от 0 до 5. Поля сведены в таблице 2.

Таблица 2 — Свойства площадки

Свойство	**Тип**	**Назначение**
id	number	Номер площадки
name	string	Название
description	string	Краткое описание
city	string	Город
adress	string	Адрес
stack	string[]	Стек людей на площадке
image	string	Путь к файлу в public
imageAlt	string	Текст, если фото не видно
latitude	number	Широта для метки
longitude	number	Долгота для метки
coords	{x, y}	Та же точка: x — долгота, y — широта
createdAt	Date	Дата появления в базе

Фотографии лежат в public/images/spots и уже использовались в каталоге. Компонент Image из next/image выводит их с fill и sizes. Альтернативный текст описывает кадр. Файлы, кадры и источники сведены в таблице 3.

Таблица 3 — Локальные фотографии и их источники

Файл	**Что на снимке**	**Источник**
tochka-kipeniya.jpg	Зал с паркетом, ряды стульев, спикер у сцены	Иллюстрация каталога. Автор и лицензия в файле не записаны
growup-ligovsky.jpg	Длинные столы у окон и люди за мониторами	Иллюстрация каталога. Автор и лицензия в файле не записаны
bench-petrogradka.jpg	Двое за ноутбуком у книжных полок	Иллюстрация каталога. Автор и лицензия в файле не записаны
sevkabel.jpg	Фасад порта: мурал, лестница, столики на площади	Wikimedia Commons, Sevkabel Port 01.jpg, автор MBH, CC BY-SA 4.0
itmo-cowork.jpg	Интерьер зала: столы, ноутбуки, доска	Иллюстрация интерьера для карточки ITMO Cowork. Автор и лицензия в файле не записаны
praktik-vo.jpg	Переговорная на двоих и стеклянная стена	Иллюстрация каталога. Автор и лицензия в файле не записаны

Файл sevkabel.jpg — копия снимка Sevkabel Port 01.jpg с Wikimedia Commons в размере 1280×960. Автор — MBH, дата съёмки 30 апреля 2024 года, лицензия CC BY-SA 4.0, страница файла https://commons.wikimedia.org/wiki/File:Sevkabel_Port_01.jpg. У остальных пяти файлов в каталоге нет записи об авторе и лицензии.

5 Галерея и модальное окно

PlacesGallery перебирает массив spots и рисует кнопку на каждую площадку. Внутри кнопки стоит next/image. Кнопка, а не div с обработчиком, получает фокус и срабатывает с клавиатуры. Выбранная площадка записывается в selected. Пока selected равен null, окна нет. Когда в selected лежит объект, рядом монтируется PhotoModal.

Окно выводится через createPortal во второй аргумент document.body. В дереве React модальное окно остаётся потомком галереи, а в DOM узел оказывается прямым потомком body и перекрывает страницу. Контекст DemoAuthProvider при этом доступен: портал меняет место узла в документе и не вырезает компонент из дерева React, поэтому useDemoAuth внутри окна читает того же пользователя.

5.1 Состояние и хуки

useState возвращает пару: текущее значение и функцию, которая его меняет. После вызова этой функции React рисует компонент заново. В галерее три таких значения. selected хранит площадку или null. favoriteIds хранит Set со строковыми номерами выбранных площадок. returnFocusTo хранит кнопку карточки, с которой открыли окно, чтобы после закрытия вернуть на неё фокус. Открытие с карты передаёт null: метка на странице не является кнопкой карточки.

useRef хранит узел, который не должен сам по себе вызывать повторный рисунок. У окна это контейнер диалога и кнопка закрытия. У меню это кнопка открытия, кнопка закрытия и панель. У карты это div, в который Яндекс Карты вставляют свой холст.

useEffect в окне при открытии прячет прокрутку documentElement, ставит фокус на кнопку закрытия и слушает Escape. Функция очистки снимает слушатель, возвращает прежнее значение overflow и вызывает focus у кнопки карточки. Та же очистка выполняется, когда окно размонтируется и когда меняются зависимости эффекта. У карты очистка вызывает destroy у

созданного экземпляра Map и помечает сборку как отменённую, чтобы поздний ответ скрипта не трогал уже снятый узел.

5.2 Доступность диалога

Контейнер окна объявлен диалогом. Заголовок h2 имеет id spot-modal-title, и aria-labelledby указывает на него. Закрытие по подложке срабатывает только когда target совпадает с currentTarget, поэтому щелчок по фотографии и тексту окно не закрывает. Tab и Shift+Tab переходят по ссылкам и кнопкам внутри диалога. С последнего элемента фокус уходит на первый, с первого при Shift+Tab — на последний. Сводка требований приведена в таблице 4.

Таблица 4 — Требования к модальному диалогу

Требование	**Как сделано**
Роль диалога	role="dialog" и aria-modal="true"
Имя окна	aria-labelledby="spot-modal-title"
Видимое закрытие	На кнопке виден знак ×, доступное имя — «Закрыть окно»
Клавиша Escape	Слушатель keydown вызывает onClose
Щелчок по подложке	Закрытие при target === currentTarget
Фокус при открытии	Фокус на кнопке закрытия
Цикл Tab	С краёв списка фокус переносится на другой край
Возврат фокуса	focus() у кнопки карточки в очистке эффекта
Блокировка прокрутки	overflow документа ставится в hidden и восстанавливается

Гость видит ссылку на /auth/login. Учебный пользователь видит кнопку избранного. Набор идентификаторов живёт в useState и пропадает при обновлении страницы. Записи в localStorage нет. Скрытая у гостя кнопка

меняет только разметку. Это условие интерфейса. Права на сервере оно не проверяет, потому что серверной операции сохранения нет: тот же Set можно было бы изменить только в памяти этой вкладки.

6 Бургер-меню

Кнопка «Открыть меню» имеет aria-expanded и aria-controls="main-menu-panel". Панель — это dialog с тем же id. Список ссылок собран через Link. usePathname сравнивает адрес с пунктом. Для «Главной» активны и /, и /start, потому что страница /start выполняет redirect на /. Остальные пункты активны на своём адресе и на вложенных сегментах. Текущая ссылка получает класс menu__link_current и aria-current="page".

Пока панель открыта, прокрутка документа заблокирована, фокус стоит на кнопке «Закрыть меню», Escape и щелчок по подложке закрывают панель, Tab ходит по её ссылкам и кнопкам. Переход по Link закрывает меню. После закрытия фокус возвращается на кнопку открытия. Гость видит «Войти». Если в контексте есть user, на этом месте имя и кнопка «Выйти»: она вызывает logout и закрывает панель.

7 Карта

PhotoMap читает process.env.NEXT_PUBLIC_YANDEX_MAPS_API_KEY. Префикс NEXT_PUBLIC_ означает, что значение попадает в код, который выполняется в браузере. Его видно в собранной странице, поэтому такой ключ нельзя считать серверным секретом. Для карты это ключ доступа к клиентскому API, его и нужно отдавать браузеру вместе со скриптом.

Скрипт https://api-maps.yandex.ru/2.1/ подключается один раз, с id pairwise-yandex-maps, параметрами apikey и lang=ru_RU. Загрузка асинхронная: документ не ждёт файл карт, чтобы нарисовать галерею. Обработчик onload вызывает ymaps.ready. Этот метод дожидается, пока API закончит свою инициализацию, и только потом разрешает создать Map и Placemark. Центр карты — [59.942, 30.3], масштаб 12. Для каждой площадки создаётся метка в точке [latitude, longitude]. В hintContent передаётся название, в balloonContent — адрес. Событие click метки вызывает onSelect, и галерея открывает то же окно, что и карточка.

Если ключа нет, состояние сразу равно no-key и скрипт не запрашивается. На странице остаётся текст «Карта не подключена: в .env.local нет ключа NEXT_PUBLIC_YANDEX_MAPS_API_KEY». Если скрипт или инициализация завершились ошибкой, показывается «Не удалось загрузить карту».

8 Результаты проверки

Команды npm run lint и npm run build завершились без ошибок. Сборка отдала страницу /gallery. Ниже экраны работающего приложения: сетка, окно гостя, бургер, действие гостя, действие после учебного входа, карта с метками и сообщение на случай, когда ключа нет.

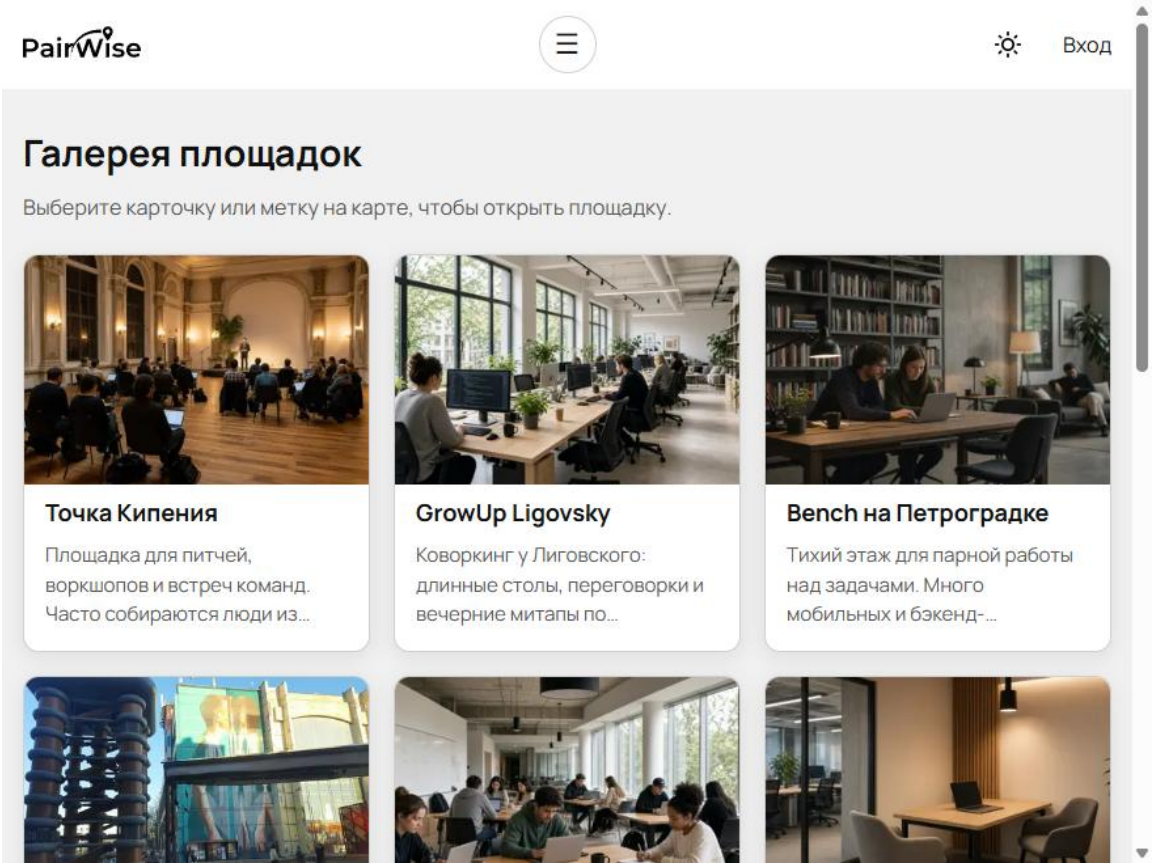


Рисунок 2 — Галерея площадок

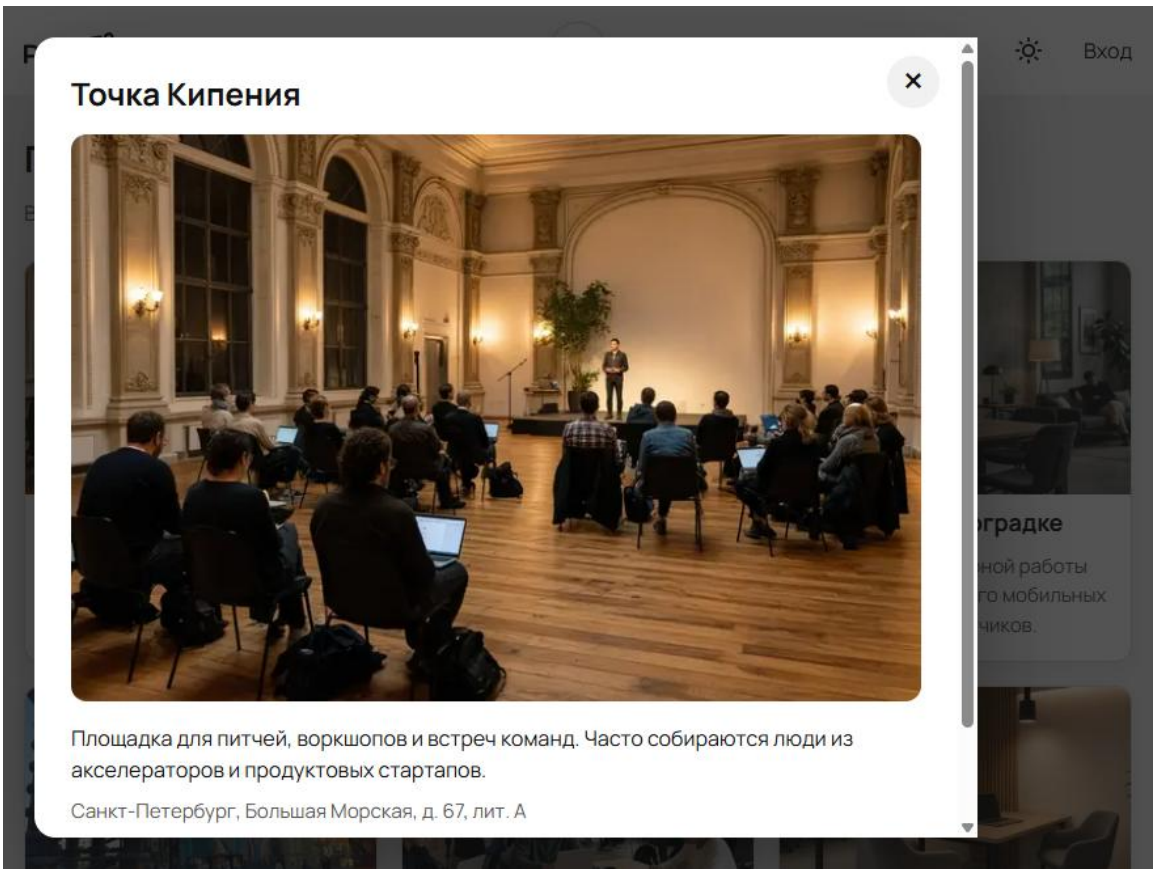


Рисунок 3 — Модальное окно «Точка Кипения»

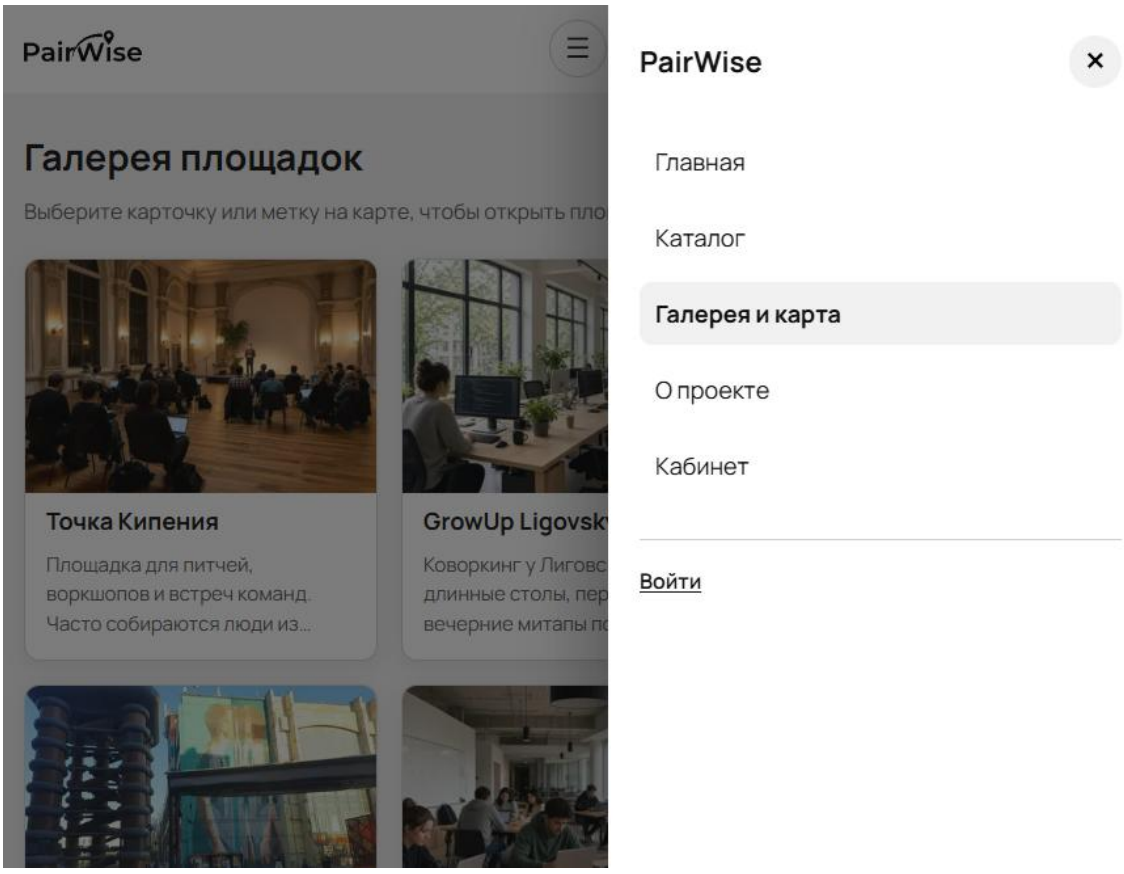


Рисунок 4 — Открытое бургер-меню

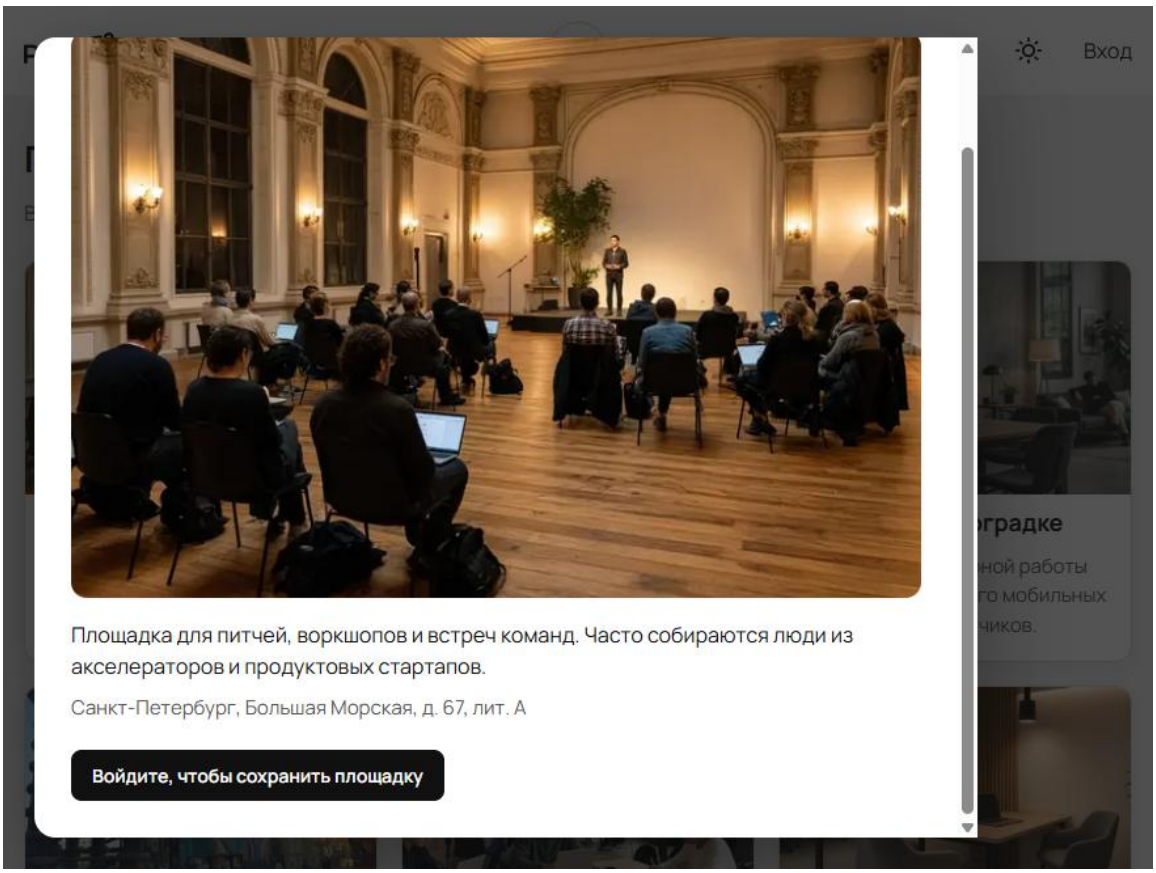


Рисунок 5 — Действие гостя в модальном окне

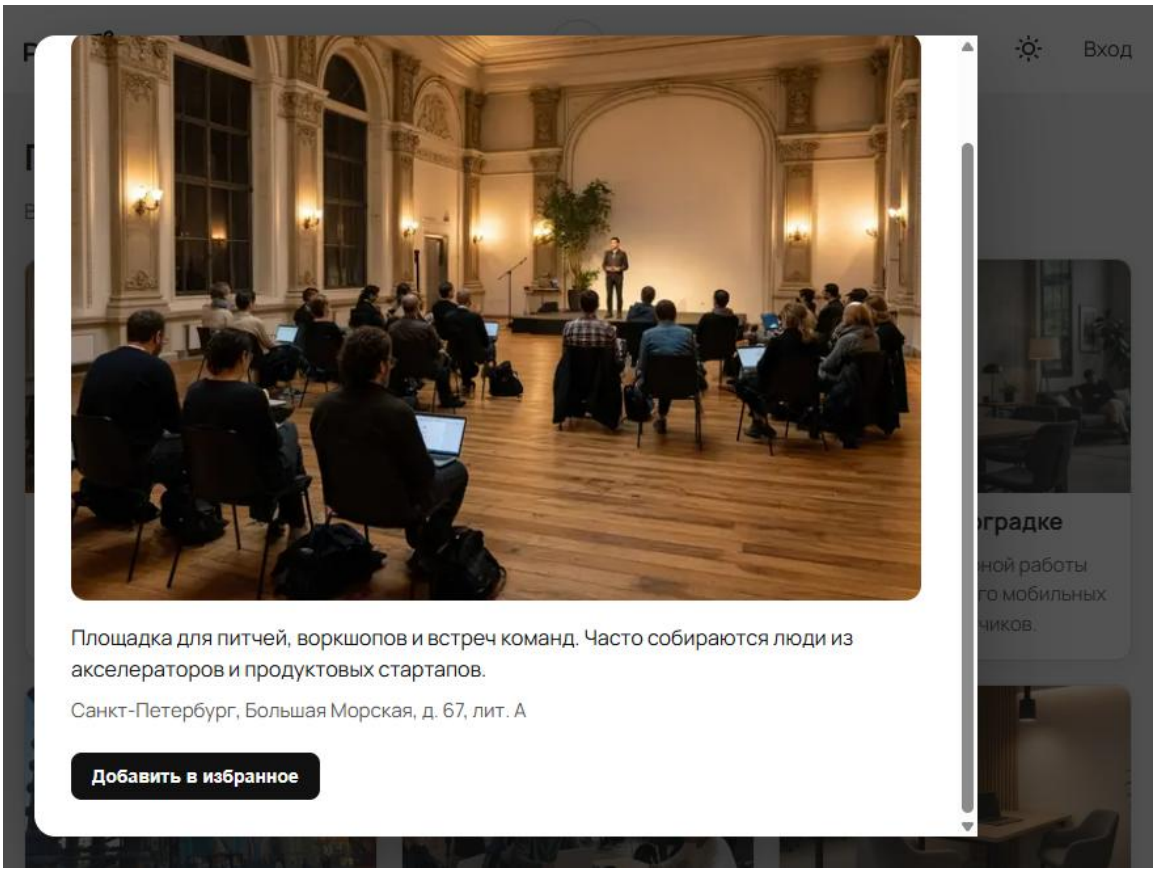


Рисунок 6 — Кнопка избранного после учебного входа

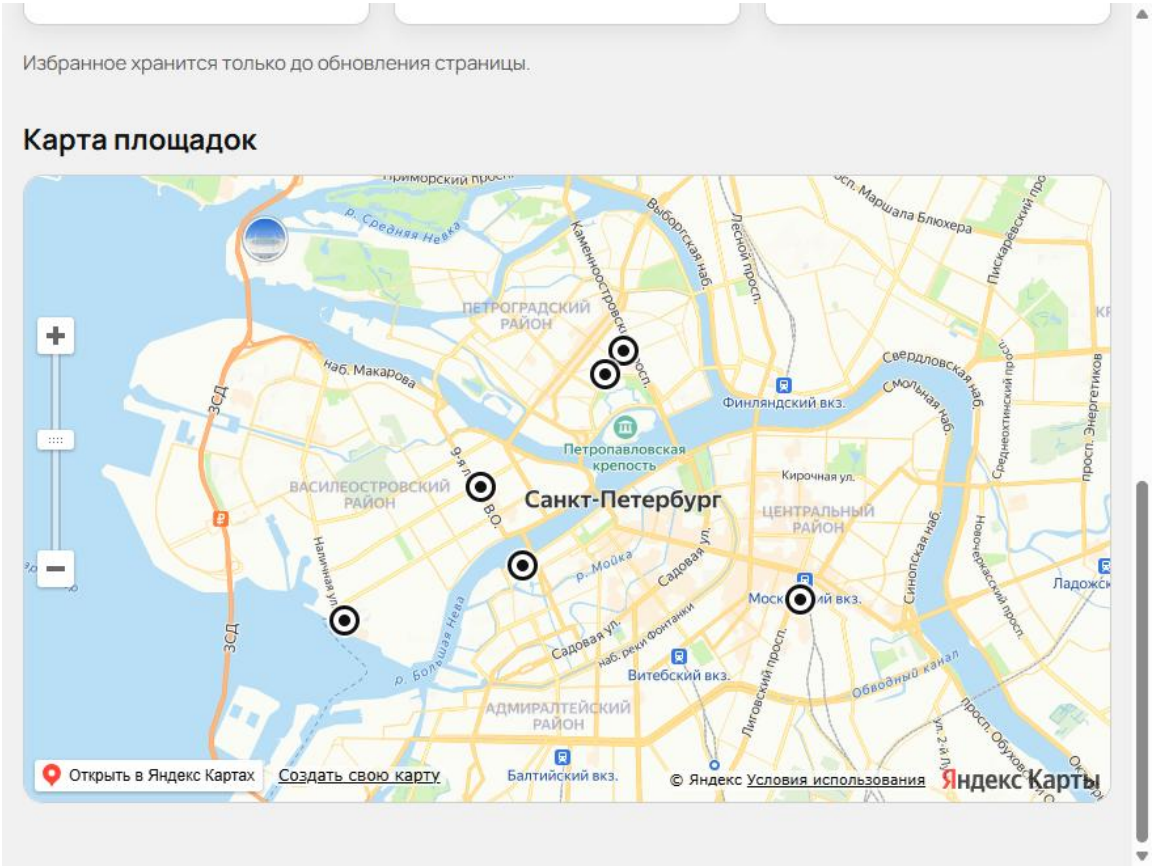


Рисунок 7 — Карта площадок с шестью метками

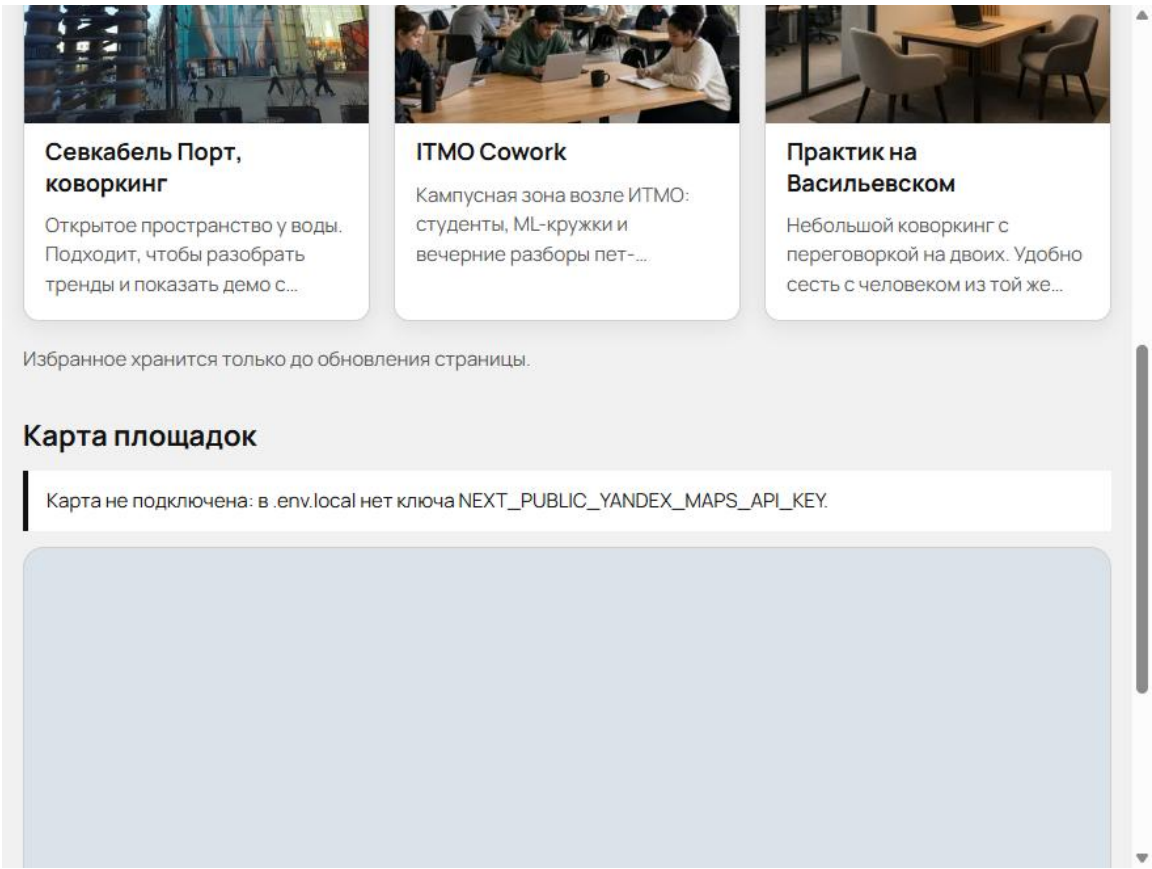


Рисунок 8 — Сообщение, если ключ карты не задан

На рисунке 2 видны шапка и карточки площадок. На рисунке 3 окно перекрывает страницу и показывает фотографию зала. На рисунке 4 текущий пункт — «Галерея и карта», внизу панели у гостя ссылка «Войти». На рисунке 5 гостю предложено войти. На рисунке 6 после входа под адресом стоит «Добавить в избранное». На рисунке 7 карта Санкт-Петербурга и шесть меток. Щелчок по метке открыл окно «ITMO Cowork». На рисунке 8 тот же блок без ключа: скрипт не загружается, виден текст про NEXT_PUBLIC_YANDEX_MAPS_API_KEY.

9 Ответы на контрольные вопросы

Для чего требуется директива «use client»?

Она помечает модуль, которому нужны состояние, события, эффекты или браузерные API. В этой работе директива стоит у галереи, окна, карты и бургер-меню.

Почему page.tsx галереи может оставаться Server Component?

Сам файл только выводит заголовок и подключает PlacesGallery. Состояние и обработчики живут в клиентском компоненте, поэтому страница маршрута остаётся серверной.

Что возвращает useState?

Пару из текущего значения и функции обновления. Вызов функции обновления приводит к новому рендеру. В галерее так устроены выбранная площадка, множество избранного и кнопка для возврата фокуса.

Для каких задач используется useRef?

Для хранения DOM-узла между рендерами без лишнего обновления состояния. Через ref берутся диалог, кнопки меню и контейнер карты.

Когда выполняется cleanup useEffect?

Перед повторным запуском эффекта и при размонтировании компонента. В окне очистка снимает Escape, возвращает прокрутку и фокус. На карте очистка уничтожает экземпляр Map.

Чем портал изменяет физическое размещение DOM?

createPortal оставляет компонент в дереве React и переносит его узел в другой контейнер. Окно монтируется в document.body и рисуется поверх страницы.

Почему контекст остаётся доступен внутри портала?

Провайдер ищется по дереву React. Портал это дерево не разрывает, поэтому PhotoModal по-прежнему читает DemoAuthProvider.

Какие атрибуты нужны контейнеру модального диалога?

role="dialog", aria-modal="true" и aria-labelledby на заголовок окна. Рядом нужна видимая кнопка закрытия.

Как должны работать Tab, Shift+Tab и Escape?

Tab и Shift+Tab циклически ходят по фокусируемым элементам внутри диалога. Escape закрывает окно.

Почему для карточки используется button, а не div с onClick?

Кнопка по умолчанию попадает в порядок Tab и активируется с клавиатуры. У div этого нет, пока ему отдельно не задать роль и обработку клавиш.

Какие задачи решает next/image?

Компонент выводит локальный файл, задаёт alt и sizes и подставляет размеры под ширину экрана. В карточках и в окне изображение занимает контейнер через fill.

Где хранятся локальные изображения?

В каталоге public/images/spots. Путь в поле image начинается с /images/spots/.

Почему API карты загружается асинхронно?

Скрипт карт нужен только блоку карты. Асинхронная вставка не задерживает разметку галереи. Страница остаётся рабочей, даже если скрипт ещё не пришёл или ключа нет.

Для чего используется ymaps.ready?

Колбэк внутри ready выполняется после инициализации API. До этого момента конструкторы Map и Placemark ещё нельзя вызывать.

Как Placemark добавляется на карту?

Для площадки создаётся ymaps.Placemark с координатами [широта, долгота], подсказкой и адресом. Метка добавляется через map.geoObjects.add. Щелчок вызывает выбор этой площадки.

Почему NEXT_PUBLIC_YANDEX_MAPS_API_KEY не является серверным секретом?

Префикс NEXT_PUBLIC_ встраивает значение в клиентский код. Браузер передаёт ключ в адресе скрипта карт, и его можно прочитать на странице.

Почему скрытие действия для гостя не защищает данные?

Условие user меняет кнопку на ссылку только в интерфейсе. Избранное и так хранится в памяти вкладки до обновления. Серверной проверки права в работе нет, поэтому скрытая кнопка не закрывает данные.

ЗАКЛЮЧЕНИЕ

В PairWise собрана клиентская часть галереи. У площадки есть локальная фотография, альтернативный текст и координаты. Страница /gallery показывает карточки, модальное окно и блок карты. Окно открывается в document.body, закрывается по Escape и по подложке, держит фокус внутри и возвращает его на карточку.

Бургер-меню заменило прежний список в шапке и отмечает текущий раздел. Гость и учебный пользователь видят в окне разные действия. Избранное живёт до обновления страницы. Карта показывает шесть меток, щелчок по метке открывает окно площадки. Без ключа в окружении на этом месте остаётся сообщение, что карта не подключена. Команды проверки и производственной сборки прошли без ошибок.

ПРИЛОЖЕНИЕ А Листинги исходного кода

Листинг А.1 — Файл app/data/spots.ts

```
export type SpotType =
    {
        id: number,
        name: string,
        description: string,
        city: string,
        adress: string,
        stack: string[],
        image: string,
        imageAlt: string,
        latitude: number,
        longitude: number,
        coords: {x: number, y: number},
        createdAt: Date,
    }

export const spots: SpotType[] = [
    {
        id: 0,
        name: "Точка Кипения",
        description: "Площадка для питчей, воркшопов и встреч команд. Часто
собираются люди из акселераторов и продуктовых стартапов.",
        city: "Санкт-Петербург",
        adress: "Большая Морская, д. 67, лит. А",
        stack: ["Product", "JavaScript", "Python"],
        image: "/images/spots/tochka-kipeniya.jpg",
        imageAlt: "Зал с паркетом и рядами стульев, люди с ноутбуками слушают
спикера у сцены",
        latitude: 59.9318,
        longitude: 30.2884,
        coords: { x: 30.2884, y: 59.9318 },
        createdAt: new Date("2023-03-12"),
    },
    {
        id: 1,
        name: "GrowUp Ligovsky",
        description: "Коворкинг у Лиговского: длинные столы, переговорки и
вечерние митапы по фронтенду.",
        city: "Санкт-Петербург",
        adress: "Лиговский проспект, д. 50, корп. 12",
        stack: ["TypeScript", "React", "Next.js"],
        image: "/images/spots/growup-ligovsky.jpg",
        imageAlt: "Длинные столы коворкинга у окон, за мониторами сидят
несколько человек",
        latitude: 59.9274,
        longitude: 30.3612,
        coords: { x: 30.3612, y: 59.9274 },
        createdAt: new Date("2023-06-01"),
    },
    {
        id: 2,
        name: "Bench на Петроградке",
        description: "Тихий этаж для парной работы над задачами. Много
мобильных и бэкенд-разработчиков.",
        city: "Санкт-Петербург",
        adress: "Каменноостровский проспект, д. 10",
        stack: ["Kotlin", "Swift", "Go"],
```

```
        image: "/images/spots/bench-petrogradka.jpg",
        imageAlt: "Двое за ноутбуком у книжных полок, на дальнем диване ещё
один человек",
        latitude: 59.9601,
        longitude: 30.3148,
        coords: { x: 30.3148, y: 59.9601 },
        createdAt: new Date("2023-09-18"),
    },
    {
        id: 3,
        name: "Севкабель Порт, коворкинг",
        description: "Открытое пространство у воды. Подходит, чтобы разобрать
тренды и показать демо с ноутбука.",
        city: "Санкт-Петербург",
        adress: "Кожевенная линия, д. 40",
        stack: ["UX", "Figma", "React"],
        image: "/images/spots/sevkabel.jpg",
        imageAlt: "Фасад Севкабель Порта с муралом, лестницей и столиками на
площади",
        latitude: 59.9246,
        longitude: 30.2419,
        coords: { x: 30.2419, y: 59.9246 },
        createdAt: new Date("2024-01-09"),
    },
    {
        id: 4,
        name: "ITMO Cowork",
        description: "Кампусная зона возле ИТМО: студенты, ML-кружки и
вечерние разборы пет-проектов.",
        city: "Санкт-Петербург",
        adress: "Кронверкский проспект, д. 49",
        stack: ["Python", "ML", "C++"],
        image: "/images/spots/itmo-cowork.jpg",
        imageAlt: "Кампусный зал с деревянными столами, ноутбуками и доской у
стены",
        latitude: 59.9571,
        longitude: 30.3101,
        coords: { x: 30.3101, y: 59.9571 },
        createdAt: new Date("2024-04-22"),
    },
    {
        id: 5,
        name: "Практик на Васильевском",
        description: "Небольшой коворкинг с переговоркой на двоих. Удобно
сесть с человеком из той же области работы.",
        city: "Санкт-Петербург",
        adress: "Средний проспект В.О., д. 36/40",
        stack: ["Java", "Spring", "DevOps"],
        image: "/images/spots/praktik-vo.jpg",
        imageAlt: "Переговорная на двоих: стол, ноутбук и стеклянная стена в
общий зал",
        latitude: 59.9423,
        longitude: 30.2776,
        coords: { x: 30.2776, y: 59.9423 },
        createdAt: new Date("2024-08-05"),
    },
]
```

Листинг А.2 — Файл app/gallery/page.tsx

```
import PlacesGallery from "./_components/PlacesGallery";
import styles from "./gallery.module.css";
```

```
const GalleryPage = () => {
    return (
        <main className={`${styles.gallery} container`}>
            <h1 className={styles.gallery__title}>Галерея площадок</h1>
            <p className={styles.gallery__lead}>
                Выберите карточку или метку на карте, чтобы открыть площадку.
            </p>
            <PlacesGallery />
        </main>
    );
};

export default GalleryPage;
```

Листинг А.3 — Файл app/gallery/_components/PlacesGallery.tsx

```
"use client";

import Image from "next/image";
import { useCallback, useState } from "react";
import { spots, type SpotType } from "../../data/spots";
import PhotoMap from "./PhotoMap";
import PhotoModal from "./PhotoModal";
import styles from "./PlacesGallery.module.css";

const PlacesGallery = () => {
    const [selected, setSelected] = useState<SpotType | null>(null);
    const [favoriteIds, setFavoriteIds] = useState<Set<string>>(() => new
Set());
    const [returnFocusTo, setReturnFocusTo] = useState<HTMLButtonElement |
null>(null);

    function openFromCard(spot: SpotType, button: HTMLButtonElement) {
        setReturnFocusTo(button);
        setSelected(spot);
    }

    const openFromMap = useCallback((spot: SpotType) => {
        setReturnFocusTo(null);
        setSelected(spot);
    }, []);

    function closeModal() {
        setSelected(null);
    }

    function toggleFavorite(id: string) {
        setFavoriteIds((current) => {
            const next = new Set(current);

            if (next.has(id)) {
                next.delete(id);
            } else {
                next.add(id);
            }

            return next;
        });
    }

    return (
```

```
        <>
            <ul className={styles.gallery__grid}>
                {spots.map((spot) => (
                    <li
                        key={spot.id}
                        className={styles.gallery__item}
                    >
                        <button
                            className={styles.gallery__card}
                            type="button"
                            onClick={(event) => openFromCard(spot,
event.currentTarget)}
                        >
                            <span className={styles.gallery__media}>
                                <Image
                                    src={spot.image}
                                    alt={spot.imageAlt}
                                    fill
                                    sizes="(max-width: 700px) 100vw, 33vw"
                                    className={styles.gallery__image}
                                />
                            </span>
                            <span
className={styles.gallery__name}>{spot.name}</span>
                            <span
className={styles.gallery__text}>{spot.description}</span>
                        </button>
                    </li>
                ))}
            </ul>

            <p className={styles.gallery__note}>
                Избранное хранится только до обновления страницы.
            </p>

            <PhotoMap
                spots={spots}
                onSelect={openFromMap}
            />

            {selected ? (
                <PhotoModal
                    spot={selected}
                    isFavorite={favoriteIds.has(String(selected.id))}
                    onToggleFavorite={() =>
toggleFavorite(String(selected.id))}
                    onClose={closeModal}
                    returnFocusTo={returnFocusTo}
                />
            ) : null}
        </>
    );
};

export default PlacesGallery;
```

Листинг А.4 — Файл app/gallery/_components/PhotoModal.tsx

```
"use client";

import Image from "next/image";
import Link from "next/link";
```

```
import { useEffect, useRef, type KeyboardEvent as ReactKeyboardEvent } from
"react";
import { createPortal } from "react-dom";
import type { SpotType } from "../../data/spots";
import { useDemoAuth } from "../../providers/useDemoAuth";
import styles from "./PhotoModal.module.css";

type PhotoModalProps = Readonly<{
    spot: SpotType;
    isFavorite: boolean;
    onToggleFavorite: () => void;
    onClose: () => void;
    returnFocusTo: HTMLElement | null;
}>;

const focusableSelector = [
    "a[href]",
    "button:not([disabled])",
    "input:not([disabled])",
    "select:not([disabled])",
    "textarea:not([disabled])",
    "[tabindex]:not([tabindex=\"-1\"])",
].join(",");

const PhotoModal = ({
    spot,
    isFavorite,
    onToggleFavorite,
    onClose,
    returnFocusTo,
}: PhotoModalProps) => {
    const { user } = useDemoAuth();
    const dialogRef = useRef<HTMLDivElement>(null);
    const closeButtonRef = useRef<HTMLButtonElement>(null);

    useEffect(() => {
        const previousOverflow = document.documentElement.style.overflow;
        document.documentElement.style.overflow = "hidden";
        closeButtonRef.current?.focus();

        function handleEscape(event: KeyboardEvent) {
            if (event.key === "Escape") {
                onClose();
            }
        }

        window.addEventListener("keydown", handleEscape);

        return () => {
            window.removeEventListener("keydown", handleEscape);
            document.documentElement.style.overflow = previousOverflow;
            returnFocusTo?.focus();
        };
    }, [onClose, returnFocusTo]);

    function keepFocusInside(event: ReactKeyboardEvent<HTMLDivElement>) {
        if (event.key !== "Tab") {
            return;
        }

        const focusable = Array.from(
```

```
dialogRef.current?.querySelectorAll<HTMLElement>(focusableSelector) ?? [],
        );

        if (focusable.length === 0) {
            event.preventDefault();
            return;
        }

        const first = focusable[0];
        const last = focusable[focusable.length - 1];

        if (event.shiftKey && document.activeElement === first) {
            event.preventDefault();
            last.focus();
        } else if (!event.shiftKey && document.activeElement === last) {
            event.preventDefault();
            first.focus();
        }
    }

    return createPortal(
        <div
            className={styles.modal__overlay}
            onMouseDown={(event) => {
                if (event.target === event.currentTarget) {
                    onClose();
                }
            }}
        >
            <div
                ref={dialogRef}
                className={styles.modal__dialog}
                role="dialog"
                aria-modal="true"
                aria-labelledby="spot-modal-title"
                onKeyDown={keepFocusInside}
            >
                <button
                    ref={closeButtonRef}
                    className={styles.modal__close}
                    type="button"
                    aria-label="Закрыть окно"
                    onClick={onClose}
                >
                    ×
                </button>
                <h2
                    id="spot-modal-title"
                    className={styles.modal__title}
                >
                    {spot.name}
                </h2>
                <div className={styles.modal__media}>
                    <Image
                        src={spot.image}
                        alt={spot.imageAlt}
                        fill
                        sizes="(max-width: 700px) calc(100vw - 3rem), 45rem"
                        className={styles.modal__image}
                    />
                </div>
                <p className={styles.modal__text}>{spot.description}</p>
```

```
                <p className={styles.modal__address}>
                    {spot.city}, {spot.adress}
                </p>
                {user ? (
                    <button
                        className={styles.modal__action}
                        type="button"
                        onClick={onToggleFavorite}
                    >
                        {isFavorite ? "Убрать из избранного" : "Добавить в
избранное"}
                    </button>
                ) : (
                    <Link
                        href="/auth/login"
                        className={styles.modal__action}
                    >
                        Войдите, чтобы сохранить площадку
                    </Link>
                )}
            </div>
        </div>,
        document.body,
    );
};

export default PhotoModal;
```

Листинг А.5 — Файл app/gallery/_components/PhotoMap.tsx

```
"use client";

import { useEffect, useRef, useState } from "react";
import type { SpotType } from "../../data/spots";
import styles from "./PhotoMap.module.css";

type Coordinates = [number, number];

type YandexPlacemark = {
    events: {
        add: (eventName: string, handler: () => void) => void;
    };
};

type YandexMap = {
    geoObjects: {
        add: (placemark: YandexPlacemark) => void;
    };
    destroy: () => void;
};

type YandexMapsApi = {
    ready: (callback: () => void) => void;
    Map: new (
        container: HTMLElement,
        state: { center: Coordinates; zoom: number; controls: string[] },
    ) => YandexMap;
    Placemark: new (
        coordinates: Coordinates,
        properties: { hintContent: string; balloonContent: string },
        options: { preset: string; iconColor: string },
    ) => YandexPlacemark;
```

```
};

declare global {
    interface Window {
        ymaps?: YandexMapsApi;
    }
}

const scriptId = "pairwise-yandex-maps";
let mapsPromise: Promise<YandexMapsApi> | null = null;

function loadMaps(apiKey: string): Promise<YandexMapsApi> {
    if (window.ymaps) {
        return new Promise((resolve) => {
            window.ymaps?.ready(() => resolve(window.ymaps as
YandexMapsApi));
        });
    }

    if (mapsPromise) {
        return mapsPromise;
    }

    mapsPromise = new Promise((resolve, reject) => {
        function resolveWhenReady() {
            if (!window.ymaps) {
                mapsPromise = null;
                reject(new Error("Yandex Maps API is unavailable"));
                return;
            }

            window.ymaps.ready(() => resolve(window.ymaps as YandexMapsApi));
        }

        const existing = document.getElementById(scriptId) as
HTMLScriptElement | null;

        if (existing) {
            existing.addEventListener("load", resolveWhenReady, { once: true
});
            existing.addEventListener("error", () => reject(new Error("Map
script error")), {
                once: true,
            });
            return;
        }

        const script = document.createElement("script");
        script.id = scriptId;
        script.src = `https://api-
maps.yandex.ru/2.1/?apikey=${encodeURIComponent(apiKey)}&lang=ru_RU`;
        script.async = true;
        script.addEventListener("load", resolveWhenReady, { once: true });
        script.addEventListener("error", () => {
            mapsPromise = null;
            reject(new Error("Map script error"));
        }, { once: true });
        document.head.appendChild(script);
    });

    return mapsPromise;
}
```

```
type PhotoMapProps = Readonly<{
    spots: SpotType[];
    onSelect: (spot: SpotType) => void;
}>;

const PhotoMap = ({ spots, onSelect }: PhotoMapProps) => {
    const containerRef = useRef<HTMLDivElement>(null);
    const apiKey = process.env.NEXT_PUBLIC_YANDEX_MAPS_API_KEY ?? "";
    const [status, setStatus] = useState<"loading" | "ready" | "error" | "no-
key">(
        apiKey ? "loading" : "no-key",
    );

    useEffect(() => {
        if (!apiKey) {
            return;
        }

        if (!containerRef.current || spots.length === 0) {
            return;
        }

        let disposed = false;
        let map: YandexMap | null = null;

        async function buildMap() {
            try {
                const ymaps = await loadMaps(apiKey);

                if (disposed || !containerRef.current) {
                    return;
                }

                map = new ymaps.Map(containerRef.current, {
                    center: [59.942, 30.3],
                    zoom: 12,
                    controls: ["zoomControl"],
                });

                spots.forEach((spot) => {
                    const placemark = new ymaps.Placemark(
                        [spot.latitude, spot.longitude],
                        {
                            hintContent: spot.name,
                            balloonContent: spot.adress,
                        },
                        {
                            preset: "islands#circleDotIcon",
                            iconColor: "#111111",
                        },
                    );

                    placemark.events.add("click", () => onSelect(spot));
                    map?.geoObjects.add(placemark);
                });

                setStatus("ready");
            } catch {
                if (!disposed) {
                    setStatus("error");
```

```
                }
            }
        }

        void buildMap();

        return () => {
            disposed = true;
            map?.destroy();
        };
    }, [apiKey, onSelect, spots]);

    return (
        <section
            className={styles.map}
            aria-labelledby="spot-map-title"
        >
            <h2
                id="spot-map-title"
                className={styles.map__title}
            >
                Карта площадок
            </h2>
            {status === "no-key" ? (
                <p className={styles.map__status}>
                    Карта не подключена: в .env.local нет ключа
NEXT_PUBLIC_YANDEX_MAPS_API_KEY.
                </p>
            ) : null}
            {status === "error" ? (
                <p className={styles.map__status}>Не удалось загрузить
карту.</p>
            ) : null}
            {status === "loading" ? (
                <p className={styles.map__status}>Карта загружается…</p>
            ) : null}
            <div
                ref={containerRef}
                className={styles.map__canvas}
                aria-label="Интерактивная карта площадок"
            />
        </section>
    );
};

export default PhotoMap;
```

Листинг А.6 — Файл app/ui/BurgerMenu/BurgerMenu.tsx

```
"use client";

import Link from "next/link";
import { usePathname } from "next/navigation";
import {
    useEffect,
    useRef,
    useState,
    type KeyboardEvent as ReactKeyboardEvent,
} from "react";
import { useDemoAuth } from "../../providers/useDemoAuth";
import styles from "./BurgerMenu.module.css";
```

```
const links = [
    { href: "/start", label: "Главная" },
    { href: "/catalog", label: "Каталог" },
    { href: "/gallery", label: "Галерея и карта" },
    { href: "/about", label: "О проекте" },
    { href: "/cabinet", label: "Кабинет" },
] as const;

const menuFocusableSelector = [
    "a[href]",
    "button:not([disabled])",
    "[tabindex]:not([tabindex=\"-1\"])",
].join(",");

function isActive(pathname: string, href: string) {
    if (href === "/start") {
        return pathname === "/" || pathname === "/start";
    }

    return pathname === href || pathname.startsWith(`${href}/`);
}

const BurgerMenu = () => {
    const pathname = usePathname();
    const { user, logout } = useDemoAuth();
    const [open, setOpen] = useState(false);
    const openButtonRef = useRef<HTMLButtonElement>(null);
    const closeButtonRef = useRef<HTMLButtonElement>(null);
    const panelRef = useRef<HTMLElement>(null);

    useEffect(() => {
        if (!open) {
            return;
        }

        const previousOverflow = document.documentElement.style.overflow;
        const openButton = openButtonRef.current;
        document.documentElement.style.overflow = "hidden";
        closeButtonRef.current?.focus();

        function handleEscape(event: KeyboardEvent) {
            if (event.key === "Escape") {
                setOpen(false);
            }
        }

        window.addEventListener("keydown", handleEscape);

        return () => {
            window.removeEventListener("keydown", handleEscape);
            document.documentElement.style.overflow = previousOverflow;
            openButton?.focus();
        };
    }, [open]);

    function closeMenu() {
        setOpen(false);
    }

    function keepFocusInside(event: ReactKeyboardEvent<HTMLElement>) {
        if (event.key !== "Tab") {
```

```
            return;
        }

        const focusable = Array.from(

panelRef.current?.querySelectorAll<HTMLElement>(menuFocusableSelector) ?? [],
        );

        if (focusable.length === 0) {
            event.preventDefault();
            return;
        }

        const first = focusable[0];
        const last = focusable[focusable.length - 1];

        if (event.shiftKey && document.activeElement === first) {
            event.preventDefault();
            last.focus();
        } else if (!event.shiftKey && document.activeElement === last) {
            event.preventDefault();
            first.focus();
        }
    }

    return (
        <>
            <button
                ref={openButtonRef}
                className={styles.menu__open}
                type="button"
                aria-label="Открыть меню"
                aria-expanded={open}
                aria-controls="main-menu-panel"
                onClick={() => setOpen(true)}
            >
                <span aria-hidden="true">☰</span>
            </button>
            {open ? (
                <div
                    className={styles.menu__overlay}
                    onMouseDown={(event) => {
                        if (event.target === event.currentTarget) {
                            closeMenu();
                        }
                    }}
                >
                    <section
                        ref={panelRef}
                        id="main-menu-panel"
                        className={styles.menu__panel}
                        role="dialog"
                        aria-modal="true"
                        aria-label="Основное меню"
                        onKeyDown={keepFocusInside}
                    >
                        <div className={styles.menu__header}>
                            <p className={styles.menu__brand}>PairWise</p>
                            <button
                                ref={closeButtonRef}
                                className={styles.menu__close}
                                type="button"
                                aria-label="Закрыть меню"
```

```
                                onClick={closeMenu}
                            >
                                ×
                            </button>
                        </div>
                        <nav aria-label="Разделы приложения">
                            <ul className={styles.menu__list}>
                                {links.map((link) => (
                                    <li key={link.href}>
                                        <Link
                                            href={link.href}
                                            className={
                                                isActive(pathname, link.href)
                                                    ? `${styles.menu__link}
${styles.menu__link_current}`
                                                    : styles.menu__link
                                            }
                                            aria-current={isActive(pathname,
link.href) ? "page" : undefined}
                                            onClick={closeMenu}
                                        >
                                            {link.label}
                                        </Link>
                                    </li>
                                ))}
                            </ul>
                        </nav>
                        <div className={styles.menu__auth}>
                            {user ? (
                                <>
                                    <span
className={styles.menu__user}>{user.name}</span>
                                    <button
                                        className={styles.menu__logout}
                                        type="button"
                                        onClick={() => {
                                            logout();
                                            closeMenu();
                                        }}
                                    >
                                        Выйти
                                    </button>
                                </>
                            ) : (
                                <Link
                                    href="/auth/login"
                                    className={styles.menu__login}
                                    onClick={closeMenu}
                                >
                                    Войти
                                </Link>
                            )}
                        </div>
                    </section>
                </div>
            ) : null}
        </>
    );
};

export default BurgerMenu;
```